AUTUMN-2025

CSE 3528– COMPILER LAB MANUAL

Department of Computer Science and Engineering (CSE)

আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম
International Islamic University Chittagong

Instructor:	**Saifur Rahaman (SR)** **Assistant Professor, Department of CSE, IIUC** **Ex-Doctoral Research Fellow, MGH, Harvard Medical School, USA** **Graduate Researcher, Broad Institute of MIT and Harvard, USA** **Doctoral Fellow, Department of Computer Science, CityU, HK** e-mail: saifurcubd@gmail.com, srahaman@iiuc.ac.bd srahaman@mgh.harvard.edu , srahaman2-c@my.cityu.edu.bd **Mobile No: +880 1815 646105**		
Time:	**Saturday, Sunday, Monday, Tuesday, Wednesday**		
Place:	**Main CSE Building, Extension Building and FAZ**		
Semester:	**5th Semester, Section 5AM, 5CM, 5CF**		
Credit Hours:	**1**	**Contact Hours:**	**2**
Co-requisite:	**CSE-3527 Compiler**		
Textbook:	**Compiler Principle and Tools (A. V. Aho, R. Sethi, JD Ullman), 2nd Ed.**		
Reference books:	• **Compiler Construction theory and practice**, William A Barret, R. M. Bates, 2nd Ed. • **Compiler Design in C**, A.J Holub • ***Flex and Bison: Text Processing Tools,*** Levine. O'Reilly Media. 2009. ISBN: 0596155972		

Course Rationale / Summary:

This compiler lab course delves into the practical aspects of compiler construction, equipping students with the skills to build programs that translate code from human-readable source program in high-level languages into target program. It familiarizes the students with hands-on knowledge and practical ideas of the construction of compilers. Students also able to gain the skills to design and implementation of the compiler for the target programming language. The competitive and standard programming ability will be expected; students should have some idea of theoretical compiler design and development skills will also be helpful. After studying this course, the students will have a practical in-depth knowledge of the design and implementation of compilers and ability to develop well-structured compiler that implements various compiler construction tools, such as the scanner, parser, code generator, and optimizer.

Learning objectives:

- To implement the Lexical Analyzer by using various Lexical Analyzer tools and C programming language
- Design and implementation of Syntax Analyzer and parser
- Experiments for NFA, DFA, CFG from given regular expression
- Develop the program to solve complex problems in compiler
- Design and implement the frontend of the compiler by means of generating Intermediate codes.
- To implement code optimization techniques.
- To implement the backend of the compiler
- To provide an Understanding of the language translation peculiarities by designing complete translator for a simple language
- Construction of the compiler for C programming language

Knowledge Profile (WK):

WK3	**Engineering fundamentals**	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline
WK4	**Specialist knowledge**	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK6	**Engineering practice**	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline

Complex Engineering Problem Solve: P1, P7

Complex Engineering Activities: N/A

Learning Domains:

Cognitive - C3: Applying, C4: Analyzing, C6: Create

Course Outcomes (COs):

Upon successful completion of this course, students will be able to:

#	CO Description	Weightage (%)
CO1	**Apply** the theoretical knowledge of compiler construction to develop a scanner, tokenizer and parser	10
CO2	**Analyze** the new tools and technologies to design and develop the modern compiler construction through hands-on experiments	30
CO3	**Develop** a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	60

Mapping of CO-PO:

#	CO Description	POs	Bloom's Taxonomy/ Domain Level	WK	WP
CO1	**Apply** the theoretical knowledge of compiler construction to develop a scanner, tokenizer and parser	PO1	C3	WK3	P1
CO2	**Analyze** the new tools and technologies to design and develop the modern compiler construction through hands-on experiments	PO5	C4	WK4	P1

CO3	**Develop** a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	PO2 PO5	C6	WK6	P1, P7

Course Grading:

Assessment Tools		Weightage (%)		
Class Attendance	-	10	10	100
Continuous evaluation and	Assignments	35	40	
	Class performances			
	Mid-term exam			
	Lab Report Book	5		
Final Exam	Quiz	10	50	
	Lab Project and report	15		
	Lab Experiments	15		
	Sessional Viva	10		
Grading Policy	As per IIUC grading policy			

Weekly Activity Plan:

Activities: Hand-on (LAB), Assignment (A), Evolution (E), Report (R), and Mid-term Exam (MT)

Week	Activities				
	LAB		**Name of Experiments**	**Assigned**	**Evaluation**
1	L1		Introduction to compiler sessional, basic C programming structures,		
2	LAB1		Implementation of comments remover of C program in C programming language	A-1	E-1
3	LAB2		Implementation of identifier and number checker of C program in C	A-2, R-1	E-2
4	LAB3		Implementation of tokenizer by using C programming language	A-3, R-2	E-3
5	L2		Basic compiler construction tools (LEX, YACC etc.) **Discussion about a simple project for compiler construction**	Project, R-3	
5	LAB4		Implementation of Lexical analyzer using by LEX tool	A-4	

6	LAB5		Implementation of Syntax analyzer using by YACC tool **Discussion Class for Mid-term Exam**	A-5, R-4	
Mid-term Exam					
7	MT		Exam based on LAB1, LAB2, LAB3, LAB4, LAB5	Viva, Report	E-4 (Exam)
Final Lab Experiments					
8	LAB6		Implementation of FIRST and FOLLOW of non-terminals from a CFG grammar	A-6, R-5	E-5
9	LAB7		Implementation of Left recursion remover in C	A-7, R-6	
10	LAB7		Implementation of left factoring remover in C	A-7, R-6	E-6
11	LAB8		Implementation of Predictive Parsing Table using FIRST and FOLLOW from a grammar	A-8, R-7	E-7
12	LAB9		Implementation of Shift-reduce Parser from a CFG	A-9, R-8	E-8
13	L4		Construction of Calculator using LEX and YACC	Project, R9	E-9 (Project)
14	LAB10		Implementation of intermediate code generation using C programming	A-10, R-10	E-10
15	L4		Lab report and project submission. Discussion and reviews for final examination		

Final Examination
(November-December 2025)

Saifur Rahaman
Assistant Professor, CSE, IIUC

Lab Requirement:
Hardware: PC [with latest Configuration]
Software: OS: Windows 7 or above, C compiler, Linux OS, YACC and LEX tools
Lab Instructions for Student:

- Students should report to the concerned lab as per the time table.
- After completion of the lab program, certification of the concerned lab instructor in the observation book is necessary.
- Student should bring a notebook and should enter the readings observations into the notebook while performing the experiment.
- The record of observations along with the detailed experimental procedure of the experiment in the immediate last session should be submitted and certified staff member in-charge.
- Students should be present in the labs for total scheduled duration.
- Students are required to prepare thoroughly to perform the experiment before coming to laboratory.

COMPILER LAB MANUAL

EXPERIMENT NO: **LAB1** **DATE: 20 SEPTEMBER 2025**

EXPERIMENT: **Implementation of comments remover of C program**

IMPLEMENTATION: **C++**

1. Objective:

To design and implement a program that removes single-line (`// ...`) and multi-line (`/* ... */`) comments from a C source file, simulating part of the lexical analysis stage in compiler construction.

To implement a C++ program that removes comments (`//` and `/*...*/`) from a C source file, while preserving comment-like sequences inside string literals and character constants**,** simulating the real behavior of a C compiler.

2. Theory:

During compilation, the **preprocessor/lexer** eliminates comments before tokenization.

- **Single-line comments**: Begin with `//` and extend until newline.
- **Multi-line comments**: Start with `/*` and end with `*/`.
- **Strings and characters**: Comment symbols inside `"..."` or `'...'` must not be treated as comments.

Thus, a proper implementation must distinguish **code context**:

1. **Normal code** – detect and remove comments.
2. **Inside string literal (`"..."`) or character constant (`'...'`)** – preserve everything as-is, including `//` or `/*`.

This requires a **state machine** approach: track whether we are inside a string, character, comment, or normal code.

3. Algorithm

1. Open input and output files.
2. Read file character by character.
3. Maintain states:
   - **NORMAL** (default)
   - **IN_STRING** (inside `"..."`)
   - **IN_CHAR** (inside `'...'`)
   - **IN_SL_COMMENT** (inside `// ...`)
   - **IN_ML_COMMENT** (inside `/* ... */`)
4. Transitions:
   - If NORMAL and `//` → switch to IN_SL_COMMENT.
   - If NORMAL and `/*` → switch to IN_ML_COMMENT.
   - If NORMAL and `"` → switch to IN_STRING.
   - If NORMAL and `'` → switch to IN_CHAR.

- If IN_STRING or IN_CHAR → copy everything until closing `"` or `'`, respecting escape characters (`\"`, `\'`).
- If IN_SL_COMMENT → skip until newline.
- If IN_ML_COMMENT → skip until closing `*/`.
- Else copy character.

5. Repeat until EOF.

4. State Transitional Diagram

States:

1. **NORMAL** – default (reading normal code)
2. **IN_STRING** – inside `"..."`
3. **IN_CHAR** – inside `'...'`
4. **IN_SL_COMMENT** – inside `// ...`
5. **IN_ML_COMMENT** – inside `/* ... */`

Transitions (Rules):

- From **NORMAL**:
  - `//` → go to **IN_SL_COMMENT**
  - `/*` → go to **IN_ML_COMMENT**
  - `"` → go to **IN_STRING**
  - `'` → go to **IN_CHAR**
  - Otherwise → stay in **NORMAL** and write character
- From **IN_STRING**:
  - If `\"` → stay in **IN_STRING** (escape sequence)
  - If `"` (not escaped) → go back to **NORMAL**
  - Otherwise → stay in **IN_STRING** and copy
- From **IN_CHAR**:
  - If `\'` → stay in **IN_CHAR** (escape sequence)
  - If `'` (not escaped) → go back to **NORMAL**
  - Otherwise → stay in **IN_CHAR** and copy
- From **IN_SL_COMMENT**:
  - If `\n` → go back to **NORMAL** (newline ends comment)
  - Otherwise → stay in **IN_SL_COMMENT** and skip
- From **IN_ML_COMMENT**:
  - If `*/` → go back to **NORMAL**
  - Otherwise → stay in **IN_ML_COMMENT** and skip

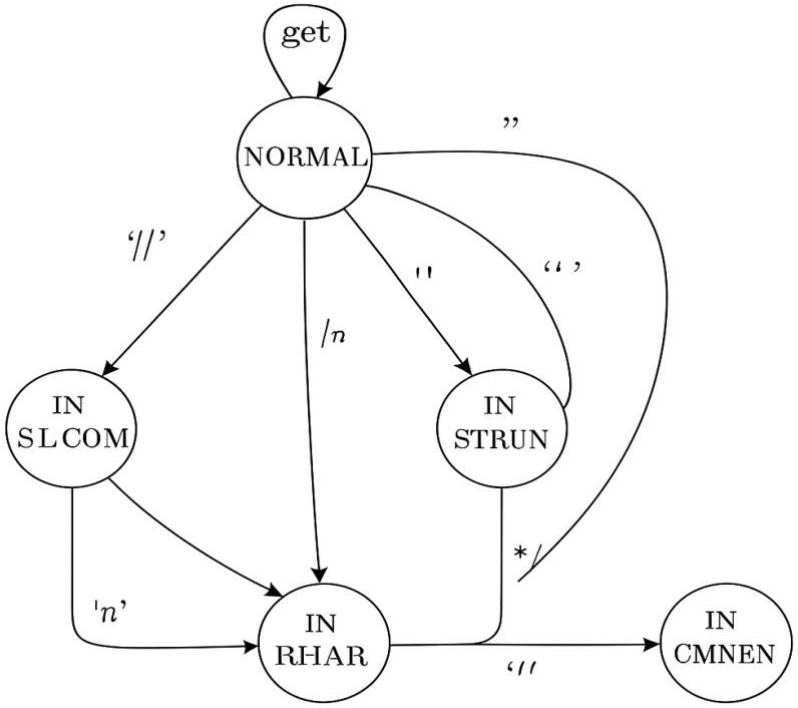

Figure 1.1: State Transitional Diagram

5. Flowchart Descriptions

1. **Start → NORMAL**
2. **Read character**
   - If `"//"` → go to **IN_SL_COMMENT**
   - If `"/*"` → go to **IN_ML_COMMENT**
   - If `"` → go to **IN_STRING**
   - If `'` → go to **IN_CHAR**
   - Else → Output character, stay in **NORMAL**
3. **IN_STRING**
   - If `"\""` → Output both, stay in **IN_STRING**
   - If `"` → Output, return to **NORMAL**
   - Else → Output character
4. **IN_CHAR**
   - If `"\'"` → Output both, stay in **IN_CHAR**
   - If `'` → Output, return to **NORMAL**
   - Else → Output character
5. **IN_SL_COMMENT**
   - If `\n` → Output newline, return to **NORMAL**
   - Else → Skip character
6. **IN_ML_COMMENT**
   - If `"*/"` → return to **NORMAL**
   - Else → Skip character
7. Repeat until **EOF → End**

6. C++ Program

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    ifstream fin("input.c");
    ofstream fout("output.c");

    if (!fin.is_open() || !fout.is_open()) {
        cout << "Error opening file!" << endl;
        return 1;
    }

    char c;
    enum State { NORMAL, IN_STRING, IN_CHAR, IN_SL_COMMENT, IN_ML_COMMENT };
    State state = NORMAL;

    while (fin.get(c)) {
        if (state == NORMAL) {
            if (c == '/') {
                if (fin.peek() == '/') {
                    state = IN_SL_COMMENT;
                    fin.get(c); // consume second '/'
                    continue;
                }
                else if (fin.peek() == '*') {
                    state = IN_ML_COMMENT;
                    fin.get(c); // consume '*'
                    continue;
                }
                else {
                    fout.put(c);
                }
            }
            else if (c == '"') {
                state = IN_STRING;
                fout.put(c);
            }
            else if (c == '\'') {
                state = IN_CHAR;
                fout.put(c);
            }
            else {
                fout.put(c);
            }
        }
        else if (state == IN_STRING) {
            fout.put(c);
            if (c == '\\') { // escape sequence
                if (fin.get(c)) fout.put(c);
            } else if (c == '"') {
                state = NORMAL;
            }
        }
        else if (state == IN_CHAR) {
            fout.put(c);
            if (c == '\\') { // escape sequence
                if (fin.get(c)) fout.put(c);
            } else if (c == '\'') {
                state = NORMAL;
            }
        }
        else if (state == IN_SL_COMMENT) {
            if (c == '\n') {
                fout.put('\n');
                state = NORMAL;
            }
        }
        else if (state == IN_ML_COMMENT) {
            if (c == '*' && fin.peek() == '/') {
                fin.get(c); // consume '/'
                state = NORMAL;
            }
```

```
            }
        }

        fin.close();
        fout.close();
        cout << "Comments removed successfully! Check output.c" << endl;
        return 0;
    }
```

7. Execution Steps

1. Save program as `remove_comments.cpp`.
2. Create `input.c` with test code.
3. Compile:
4. `g++ remove_comments.cpp -o remove_comments`
5. Run:
6. `./remove_comments`
7. Open `output.c` for cleaned code.

8. Sample Input (input.c)

```
#include <stdio.h>

int main() {
    // This is a real comment
    printf("This is not a comment: // inside string\n");
    printf("Multi-line style too: /* not a comment */\n");
    char c = '/';  // Another comment
    return 0;
}
```

9. Sample Output (output.c)

```
#include <stdio.h>

int main() {

    printf("This is not a comment: // inside string\n");
    printf("Multi-line style too: /* not a comment */\n");
    char c = '/';
    return 0;
}
```

10. Result

The C++ program successfully removed both single-line and multi-line comments, while preserving strings and characters that contained comment-like symbols.

11. Conclusion

This experiment shows a more **accurate simulation of compiler behavior**:

- True comments were removed.

- Strings and characters with `//` or `/*...*/` remained unchanged.
  This method closely models the lexical analyzer in C/C++ compilers.

IMPLEMENTATION: C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp_in, *fp_out;
    char c, next;

    fp_in = fopen("input.c", "r");
    fp_out = fopen("output.c", "w");

    if (!fp_in || !fp_out) {
        printf("Error opening file!\n");
        return 1;
    }

    while ((c = fgetc(fp_in)) != EOF) {
        if (c == '/') {
            next = fgetc(fp_in);

            // Handle single-line comment
            if (next == '/') {
                while ((c = fgetc(fp_in)) != EOF && c != '\n');
                fputc('\n', fp_out);
            }
            // Handle multi-line comment
            else if (next == '*') {
                while ((c = fgetc(fp_in)) != EOF) {
                    if (c == '*' && (next = fgetc(fp_in)) == '/')
                        break;
                }
            }
            // Not a comment, write both
            else {
                fputc(c, fp_out);
                fputc(next, fp_out);
            }
        }
        else {
            fputc(c, fp_out);
        }
    }

    fclose(fp_in);
    fclose(fp_out);

    printf("Comments removed successfully! See output.c\n");
    return 0;
}
```

IMPLEMENTATION: **LEX**

1. Objective

To design and implement a Lex program that removes single-line (`//...`) and multi-line (`/*...*/`) comments from a C source code file, demonstrating the role of lexical analysis in compiler construction.

2. Theory

In compiler construction, the **lexical analysis phase** (or scanner) is responsible for reading the source code and breaking it into tokens. Tokens represent meaningful elements such as keywords, identifiers, operators, and symbols.

Comments are present in the source code for human readability but are **not required by the compiler**. Thus, the lexical analyzer must **ignore or remove comments**.

- **Single-line comments** in C begin with `//` and continue until the end of the line.
- **Multi-line comments** start with `/*` and end with `*/`.

By using **Lex/Flex**, we can define regular expressions that match these comment patterns and instruct the lexer to ignore them. This simulates how a compiler discards comments during the lexical analysis stage.

3. Algorithm

1. Start the program and initialize the Lex analyzer.
2. Read the source code character by character.
3. If the pattern matches:
    - `//.*` → Single-line comment → Ignore until newline.
    - `/* ... */` → Multi-line comment → Ignore until `*/`.
4. If the character does not belong to a comment, print it to the output.
5. Continue until End of File (EOF).
6. End the program.

4. Lex Program

```
%{
#include <stdio.h>
%}

%%
"//".*                    ;   // Ignore single-line comment
"/*"([^*]|\*+[^/])*\*+"/" ;   // Ignore multi-line comment
.                         { printf("%s", yytext); }  // Print other
characters
\n                        { printf("\n"); }          // Preserve
newlines
%%

int main() {
    yylex();
    return 0;
}
```

5. Execution Steps

1. Save the program as `remove_comments.l`.
2. Open terminal and run the following commands:
3. `flex remove_comments.l`
4. `gcc lex.yy.c -o remove_comments`
5. `./remove_comments < input.c > output.c`
6. `input.c` → contains original C program with comments.
7. `output.c` → generated file without comments.

6. Sample Input

File: `input.c`

```
#include <stdio.h>
// This is a comment
int main() {
    printf("Hello"); /* Print hello */
    return 0; // End
}
```

7. Sample Output

File: `output.c`

```
#include <stdio.h>
int main() {
    printf("Hello");
    return 0;
}
```

8. Result

The program successfully removed both single-line and multi-line comments from the given C source code, leaving the code structure intact.

9. Conclusion

This experiment demonstrated how Lex can be used to perform lexical analysis tasks in compiler construction. We implemented a comment remover that skips over comment patterns using regular expressions. This illustrates the comment-handling functionality of a real compiler lexer and strengthens understanding of compiler design concepts.

IMPLEMENTATION: **PYTHON**

1. Objective

To implement a Python program that removes single-line (`//...`) and multi-line (`/*...*/`) comments from a C source code file. This simulates the process of ignoring comments during lexical analysis in compiler construction.

2. Theory

In compiler construction, the **lexical analyzer** reads source code character by character and converts it into tokens. Tokens are meaningful sequences like keywords, identifiers, operators, etc.

- Comments are ignored by the compiler because they do not affect program execution.
- **Single-line comments** start with `//` and end with newline (`\n`).
- **Multi-line comments** start with `/*` and end with `*/`.

Using Python, we can process the input file as a string, detect these comment patterns with regular expressions (regex), and remove them efficiently.

3. Algorithm

1. Open the input C file and read its contents as text.
2. Use regex to match and remove:
    - `//.*` → single-line comments.
    - `/\*.*?\*/` → multi-line comments.
3. Write the processed text (without comments) into an output file.
4. Display a success message.

4. Python Program

```
import re

# Open and read the C source program
with open("input.c", "r") as f:
    code = f.read()

# Regex pattern for comments
pattern = r"//.*?$|/\*.*?\*/"
# Remove comments (supports multiline)
clean_code = re.sub(pattern, "", code, flags=re.DOTALL | re.MULTILINE)

# Save output to new file
with open("output.c", "w") as f:
    f.write(clean_code)

print("Comments removed successfully! See output.c")
```

5. Execution Steps

1. Save the Python script as `remove_comments.py`.
2. Create an input file `input.c` with C code containing comments.
3. Run the program:
4. `python3 remove_comments.py`
5. Open `output.c` to view the code without comments.

6. Sample Input (input.c)

```
#include <stdio.h>

// This is a single-line comment
int main() {
    printf("Hello World"); /* Print hello */
    return 0; // End
}
```

7. Sample Output (output.c)

```
#include <stdio.h>

int main() {
    printf("Hello World");
    return 0;
}
```

8. Result

The Python program successfully removed both single-line and multi-line comments from the given C source program. The final output preserved the original program's structure but excluded comments.

9. Conclusion

This experiment demonstrates how Python can be used to simulate lexical analysis tasks in compiler construction. By applying regular expressions, the program effectively removed comments from a C source file. This illustrates how a compiler ignores comments during the scanning phase.

EXPERIMENT NO: **LAB2** **DATE: 20 SEPTEMBER 2025**

EXPERIMENT: **Implementation of Identifier and Number Checker for C Program**

IMPLEMENTATION: **C++**

1. Objective:

To implement a lexical analyzer module that checks whether a given token in a C program is a **valid identifier** or a **valid number**. The implementation is done using **C++**, **Python**, and **Lex (Flex)**.

2. Theory

In compiler design, lexical analysis is the first phase of compilation, where the source program is scanned and broken into tokens. Among the most important tokens are **identifiers** (variable names, function names) and **numbers** (integer/real constants).

Identifier Rules (C Language):

- Must begin with a letter (A–Z, a–z) or underscore (_).
- Followed by letters, digits (0–9), or underscore.
- Cannot be a reserved keyword.
- Examples: `x`, `_temp1`, `value123` ✅ ; `1abc`, `#var` ❌

Number Rules:

- Consist of digits (0–9).
- Can be integer (`123`), floating-point (`12.34`).
- Exponential notation allowed (`3.5e10`).
- Examples: `42`, `0.99`, `1e5` ✅ ; `12a`, `..23` ❌

Thus, we implement a **Finite State Automaton (FSA)** to verify tokens.

3. Algorithm

1. Read the input string (token).
2. If first character is a letter or _ → process as **Identifier** candidate.
    - Check remaining characters are letters, digits, or _.
    - If valid → Output: *Valid Identifier*.
3. Else if first character is a digit → process as **Number** candidate.
    - Check all characters follow numeric format (integer, decimal, exponent).
    - If valid → Output: *Valid Number*.
4. Else → Not a valid identifier or number.

4. Programs

(a) C++ Implementation

```
#include <iostream>
#include <cctype>
#include <string>
using namespace std;

bool isIdentifier(const string& s) {
    if (!(isalpha(s[0]) || s[0] == '_')) return false;
    for (int i = 1; i < s.size(); i++) {
        if (!(isalnum(s[i]) || s[i] == '_')) return false;
```

```
    }
    return true;
}

bool isNumber(const string& s) {
    bool dotSeen = false, expSeen = false;
    int i = 0;
    if (s[i] == '+' || s[i] == '-') i++;
    for (; i < s.size(); i++) {
        if (isdigit(s[i])) continue;
        else if (s[i] == '.' && !dotSeen) dotSeen = true;
        else if ((s[i] == 'e' || s[i] == 'E') && !expSeen) {
            expSeen = true;
            if (i+1 < s.size() && (s[i+1]=='+' || s[i+1]=='-')) i++;
        }
        else return false;
    }
    return true;
}

int main() {
    string token;
    cout << "Enter a token: ";
    cin >> token;

    if (isIdentifier(token)) cout << token << " is a valid Identifier\n";
    else if (isNumber(token)) cout << token << " is a valid Number\n";
    else cout << token << " is Invalid\n";

    return 0;
}
```

(b) Python Implementation

```
import re

def is_identifier(s):
    return re.match(r'^[A-Za-z_][A-Za-z0-9_]*$', s) is not None

def is_number(s):
    return re.match(r'^[+-]?(\d+(\.\d*)?|\.\d+)([eE][+-]?\d+)?$', s) is not None

token = input("Enter a token: ")

if is_identifier(token):
    print(f"{token} is a valid Identifier")
elif is_number(token):
    print(f"{token} is a valid Number")
else:
    print(f"{token} is Invalid")
```

(c) Lex (Flex) Program

```
%{
#include <stdio.h>
%}

IDENTIFIER [a-zA-Z_][a-zA-Z0-9_]*
NUMBER     [0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?

%%
{IDENTIFIER}  { printf("%s is a valid Identifier\n", yytext); }
{NUMBER}      { printf("%s is a valid Number\n", yytext); }
.|\n          { printf("%s is Invalid\n", yytext); }
%%

int main() {
    yylex();
    return 0;
}
```

5. Sample Output

Input:

```
myVar
12.5
1ab
3e10
```

Output:

```
myVar is a valid Identifier
12.5 is a valid Number
1ab is Invalid
3e10 is a valid Number
```

---

5. Conclusion

The experiment successfully demonstrated the lexical analysis step for recognizing **Identifiers** and **Numbers** in C language using **C++**, **Python**, and **Lex**. These modules form the foundation of compiler front-end design.

EXPERIMENT NO: **LAB3** **DATE: 1 NOVEMBER 2025**

EXPERIMENT: **Implementation of a C Tokenizer**

IMPLEMENTATION: **C++, PYTHON, AND LEX**

Objective

To implement a **C Tokenizer** that identifies and classifies all tokens in a C source program — including **keywords, identifiers, numeric constants (with exponent), character and string constants, operators, symbols, and others** — using three approaches:

1. Manual implementation in **C++** (without regular expressions)
2. Automated implementation using **Python (regex-based)**
3. Lexical analysis tool implementation using **Lex/Flex**

Theory

A **token** is the smallest unit of a program recognized by the compiler.
During the **lexical analysis phase** of compilation, the source code is scanned and divided into tokens that represent keywords, identifiers, literals, operators, and punctuators.

Types of Tokens in C

Token Type	Description	Example
Keywords	Reserved words defined by C	`int`, `return`, `for`
Identifiers	User-defined names	`sum`, `_value`, `main`
Constants	Numeric, character, or string values	`10`, `3.14e2`, `'A'`, `"Hello"`
Operators	Arithmetic, logical, relational, bitwise	`+`, `-`, `==`, `&&`, `++`
Symbols/Punctuators	Syntax elements	`{`, `}`, `;`, `(`, `)`

Algorithm (Common Logic)

1. Read the input C source code.
2. Remove comments and whitespace.
3. Identify lexemes using character-based or pattern-based parsing.
4. Classify lexemes as tokens:
    - Keywords (check against reserved list)
    - Identifiers (alphabetic or _, followed by alphanumerics)
    - Numeric constants (integers, floating, exponential)
    - String/Char constants (within quotes)
    - Operators (single/double character combinations)
    - Symbols (punctuators)
5. Store each token type in its respective collection (e.g., vector/set).
6. Print token categories in tabular format.

Implementation in C++:

This implementation simulates the lexical analyzer of a C compiler manually.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <set>
#include <cctype>
#include <iomanip>
using namespace std;

// ---------- Keyword List ----------
const string keywords[] = {
    "auto","break","case","char","const","continue","default","do","double","else",
 "enum","extern","float","for","goto","if","int","long","register","return",
 "short","signed","sizeof","static","struct","switch","typedef","union","unsigned",
    "void","volatile","while","inline","restrict","_Bool","_Complex","_Imaginary"
};
const int keywordCount = sizeof(keywords)/sizeof(keywords[0]);

// ---------- Token Sets ----------
set<string> keywordSet, identifierSet, numConstSet, charStrConstSet, operatorSet,
symbolSet, othersSet;

// ---------- Helper Functions ----------
bool isKeyword(const string &word) {
    for (int i = 0; i < keywordCount; ++i)
        if (word == keywords[i]) return true;
    return false;
}
bool isSymbol(char ch) {
    string symbols = ";,{}()[]#:";
    return symbols.find(ch) != string::npos;
}
bool isIdentifier(const string &word) {
    if (!isalpha(word[0]) && word[0] != '_') return false;
    for (char c : word)
        if (!isalnum(c) && c != '_') return false;
    return true;
}
bool isNumericConstant(const string &num) {
    bool hasDecimal = false, hasExp = false;
    int i = 0;
    if (num[i] == '+' || num[i] == '-') i++;
    for (; i < num.size(); ++i) {
        if (isdigit(num[i])) continue;
        else if (num[i] == '.' && !hasDecimal && !hasExp) hasDecimal = true;
        else if ((num[i] == 'e' || num[i] == 'E') && !hasExp) {
            hasExp = true;
            if (i+1 < num.size() && (num[i+1] == '+' || num[i+1] == '-')) i++;
        } else return false;
    }
    return true;
}
bool isCharOrStringConst(const string &token) {
    if (token.size() >= 2) {
        if ((token.front() == '"' && token.back() == '"') ||
            (token.front() == '\'' && token.back() == '\''))
            return true;
    }
    return false;
}
bool isOperatorChar(char ch) {
    string opchars = "+-*/%=<>!&|^~";
    return opchars.find(ch) != string::npos;
}
```

```
void processToken(const string &tok) {
    if (tok.empty()) return;
    if (isKeyword(tok)) keywordSet.insert(tok);
    else if (isIdentifier(tok)) identifierSet.insert(tok);
    else if (isNumericConstant(tok)) numConstSet.insert(tok);
    else if (isCharOrStringConst(tok)) charStrConstSet.insert(tok);
    else othersSet.insert(tok);
}

void tokenize(const string &code) {
    string token;
    for (size_t i = 0; i < code.size(); ++i) {
        char ch = code[i];
        if (ch == '"' || ch == '\'') {
            char quote = ch;
            string lit; lit += ch;
            i++;
            while (i < code.size()) {
                lit += code[i];
                if (code[i] == quote && lit[lit.size()-2] != '\\') break;
                i++;
            }
            charStrConstSet.insert(lit);
            token.clear();
        } else if (isOperatorChar(ch)) {
            string op(1, ch);
            if (i+1 < code.size()) {
                string two = op + code[i+1];
                if (two == "++" || two == "--" || two == "==" || two == "!=" ||
                    two == ">=" || two == "<=" || two == "&&" || two == "||" ||
                    two == ">>" || two == "<<") {
                    operatorSet.insert(two); i++; continue;
                }
            }
            operatorSet.insert(op);
        } else if (isSymbol(ch)) {
            symbolSet.insert(string(1, ch));
            processToken(token);
            token.clear();
        } else if (isspace(ch)) {
            processToken(token);
            token.clear();
        } else token += ch;
    }
    processToken(token);
}

void printTokens() {
    cout << left << setw(30) << "Token Type" << "| Token(s)" << endl;
    cout << string(60, '-') << endl;
    auto printSet = [&](string name, set<string> s) {
        cout << left << setw(30) << name << "| ";
        for (auto &x : s) cout << x << " ";
        cout << endl;
    };
    printSet("Keywords", keywordSet);
    printSet("Identifiers", identifierSet);
    printSet("Numeric Constants", numConstSet);
    printSet("Char/String Constants", charStrConstSet);
    printSet("Operators", operatorSet);
    printSet("Symbols", symbolSet);
    printSet("Others", othersSet);
}

int main() {
    ifstream file("input.c");
    if (!file.is_open()) {
        cout << "Error: input.c not found!" << endl;
```

```
        return 1;
    }
    stringstream buffer; buffer << file.rdbuf();
    tokenize(buffer.str());
    printTokens();
}
```

Implementation B — Python (Simplified Version using Regex)

```
import re

code = open("input.c").read()

keywords =
{"int","float","if","else","return","char","double","for","while","do","void"}

pattern = {
    "Keyword": r"\b(?:int|float|if|else|return|char|double|for|while|do|void)\b",
    "Identifier": r"\b[A-Za-z_]\w*\b",
    "Number": r"\b\d+(\.\d+)?(e[+-]?\d+)?\b",
    "Char/String": r"\".*?\"|'.*?'",
    "Operator": r"[+\-*/%=<>!&|^~]+",
    "Symbol": r"[;{},()#\[\]]"
}

for typ, pat in pattern.items():
    tokens = re.findall(pat, code)
    if tokens:
        print(f"{typ}: {set(tokens)}")
```

Implementation in Lex/Flex:

```
%{
#include <stdio.h>
%}

%%
"/*"([^*]|\*+[^/])*\*+"/"      ;
"//".*                          ;
[0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?    { printf("NUMBER: %s\n", yytext); }
\"([^\\\"]|\\.)*\"              { printf("STRING: %s\n", yytext); }
\'([^\\\']|\\.)*\'              { printf("CHAR: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]*          { printf("IDENTIFIER: %s\n", yytext); }
[+\-*/%=<>!&|^~]+               { printf("OPERATOR: %s\n", yytext); }
[;{},()#\[\]]                   { printf("SYMBOL: %s\n", yytext); }
[ \t\n]+                        ;
.                               { printf("OTHER: %s\n", yytext); }
%%

int main() { yylex(); return 0; }
```

Compile & Run:

```
flex tokenizer.l
gcc lex.yy.c -o tokenizer
./tokenizer < input.c
```

Sample Input (input.c):

```
int main() {
   float x = 3.14e2;
   char c = 'A';
   printf("Value: %f", x);
   return 0;
}
```

Sample Output

Token Type	Tokens
Keywords	int float char return
Identifiers	main x c printf
Numeric Constants	3.14e2
Char/String Constants	'A' "Value: %f"
Operators	= ( ) ; ,
Symbols	{ }
Others	—

Conclusion

This experiment demonstrates how a compiler's **lexical analysis phase** identifies and classifies tokens from source code.
Implementations in **C++**, **Python**, and **Lex** provide hands-on understanding of how lexical analyzers work — from manual logic to pattern-based tokenization using tools.
It also emphasizes **token categorization**, **redundancy removal**, and **language syntax handling**, essential in building compilers and interpreters.

EXPERIMENT NO: **LAB4** **DATE: 15 NOVEMBER 2025**

EXPERIMENT: **Implementation of Arithmetic Desk Calculator Using LEX (Flex)**

IMPLEMENTATION: **LEX (Flex)**

Objective

To implement a **desk calculator** capable of evaluating arithmetic expressions using **Lex**.
The calculator accepts +, –, *, /, parentheses, and real numbers.

Lex is used for tokenization **and** evaluation using C code inside the Lex file.

Theory

A typical expression grammar:

E → E + E **| E - E** **| E * E** **| E / E** **| (E)** **| NUM**	**E → E + T | E - T | T** **T → T * F | T / F | F** **F → (E) | NUM**

Lex alone cannot process context-free grammar.
But we can:

✓ Tokenize numbers & operators
✓ Use a **stack** to evaluate expressions based on operator precedence
✓ Use the *shunting-yard algorithm* inside Lex
✓ Produce evaluated result

This creates a working **desk calculator**.

Algorithm:

Shunting Yard + Stack Evaluation

Step 1: Read input expression from Lex.

Step 2: Tokenize: NUM, '+', '-', '*', '/', '(', ')'.

Step 3: Push numbers onto value stack.

Step 4: Push operators onto operator stack with precedence rules:

* and / have higher precedence than + and -.

Step 5: When closing parenthesis):

Evaluate until '(' is found.

Step 6: At end of input:

Evaluate remaining operators.

Step 7: Print final value.

Implementation in Lex (Flex):

Create a file named desk_calculator.l:

```
%{
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX 100

double val_stack[MAX];
char op_stack[MAX];
int val_top = -1, op_top = -1;

void push_val(double v) { val_stack[++val_top] = v; }
void push_op(char c) { op_stack[++op_top] = c; }

double pop_val() { return val_stack[val_top--]; }
char pop_op() { return op_stack[op_top--]; }

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

void apply_op() {
    double b = pop_val();
    double a = pop_val();
    char op = pop_op();

    if (op == '+') push_val(a + b);
    else if (op == '-') push_val(a - b);
    else if (op == '*') push_val(a * b);
    else if (op == '/') push_val(a / b);
}
%}

%%

[0-9]+(\.[0-9]+)?   {
                        push_val(atof(yytext));
                    }

[+\-*/]             {
                        while (op_top >= 0 &&
                               precedence(op_stack[op_top]) >=
precedence(yytext[0])) {
                            apply_op();
                        }
                        push_op(yytext[0]);
```

```
                    }

"("                 { push_op('('); }

")"                 {
                        while (op_top >= 0 && op_stack[op_top] != '(')
                            apply_op();
                        pop_op(); /* pop '(' */
                    }

[\t\r ]+            ;   /* ignore whitespace */

\n                  {
                        while (op_top >= 0) apply_op();
                        printf("Result = %lf\n", pop_val());
                        val_top = op_top = -1;
                    }

.                   { /* ignore any unknown char */ }

%%

int main() {
    printf("Desk Calculator (type expression, press Enter):\n");
    yylex();
    return 0;
}
```

Sample Input & Output

Input:

```
3 + 4 * 2

(10 + 2) * 3 - 4 / 2
```

Output:

```
Result = 11.000000

Result = 32.000000
```

How to Run with Flex:

✓ Linux Installation & Execution

Install Flex:

```
sudo apt-get install flex
```

Compile and run:

```
flex calculator.l
gcc lex.yy.c -o calc -lfl

./calc
```

✓ Windows (Using WinFlex-Bison):

Download:

👉 https://github.com/lexxmark/winflexbison/releases

Run Flex, compile and output:

```
flex calculator.l
gcc lex.yy.c -o calc.exe
calc.exe
```

Viva Questions Sample:

1. Why can't Lex directly parse CFGs?
2. How is the Shunting Yard algorithm used here?
3. What is operator precedence?
4. Why do we need a value stack and operator stack?
5. How does Lex return tokens?

Conclusion:

The experiment demonstrates how **Lex** can be used to construct a working **desk calculator**, even though Lex alone cannot parse CFGs. Using a **stack-based evaluation** approach embedded inside Lex rules, the calculator successfully evaluates arithmetic expressions with correct precedence and associativity.

EXPERIMENT NO: **LAB5** **DATE: 22 NOVEMBER 2025**

EXPERIMENT: **Implementation of Arithmetic Desk Calculator (Standard LL/LR Calculator) using LEX and YACC**

IMPLEMENTATION: **LEX (Flex) and YACC (Bison)**

Objective

To implement a **desk calculator** capable of evaluating arithmetic expressions using **Lex and YAAC**. The calculator accepts +, –, *, /, parentheses, and real numbers.

- **Lex (Flex)** is used for tokenization **and** evaluation using C code inside the Lex file
- **YACC (Bison)** for parsing using CFG

Theory

A typical expression grammar:

E → E + E | E - E | E * E | E / E | (E) | NUM	E → E + T | E - T | T T → T * F | T / F | F F → (E) | NUM

Using **YACC**, we can implement a calculator directly from the CFG:

YACC ensures:

- Operator precedence
- Left associativity
- Parsing using LALR(1)

Lex simply produces tokens like:

- NUM
- + - * /
- ()

Lex File implementation:

Create Yacc_desk_calcolator.l:

```
%{
#include "calc.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+(\.[0-9]+)?    { yylval.fval = atof(yytext); return NUM; }
[\t ]+               ;       /* ignore whitespaces */
\n                   { return EOL; }
```

```
"+"                     { return '+'; }
"-"                     { return '-'; }
"*"                     { return '*'; }
"/"                     { return '/'; }
"("                     { return '('; }
")"                     { return ')'; }

.                       { return yytext[0]; }

%%
int yywrap() {
return 1;
}
```

YACC / BISON File:

Create Yacc_desk_calcolator.y:

```
%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *msg);
int yylex(void);
%}

%union {
    double fval;
}

%token <fval> NUM
%token EOL

%left '+' '-'
%left '*' '/'
%right UMINUS

%type <fval> expr

%%
input:
      /* empty */
    | input expr EOL    { printf("Result = %lf\n", $2); }
    ;

expr:
      expr '+' expr     { $$ = $1 + $3; }
    | expr '-' expr     { $$ = $1 - $3; }
    | expr '*' expr     { $$ = $1 * $3; }
    | expr '/' expr     { $$ = $1 / $3; }
```

```
    | '-' expr %prec UMINUS { $$ = -$2; }
    | '(' expr ')'       { $$ = $2; }
    | NUM                { $$ = $1; }
    ;

%%

void yyerror(const char *msg) {
    fprintf(stderr, "Error: %s\n", msg);
}

int main() {
    printf("Desk Calculator (Lex + YACC)\nEnter expression:\n");
    yyparse();
    return 0;
}
```

Compile & Run:

✓ LINUX / Ubuntu

Install tools:

```
sudo apt install flex bison gcc
```

Compile and run:

```
bison -d calc.y
flex calc.l
gcc calc.tab.c lex.yy.c -o calc -lfl
./calc
```

✓ WINDOWS (WinFlexBison):

Download:
https://github.com/lexxmark/winflexbison/releases

Compile and run:

```
win_bison.exe -d calc.y
win_flex.exe calc.l
gcc calc.tab.c lex.yy.c -o calc.exe
calc.exe
```

Input:

```
3 + 4 * 2
(5 + 5) * (2 + 3)
-3 + 7
```

Output:

```
Result = 11.000000

Result = 50.000000

Result = 4.000000
```

Viva Questions Sample:

1. Why do we use YACC instead of only Lex?
2. What is the meaning of `%left` and `%right`?
3. Why is `UMINUS` needed?
4. Difference between LEX tokenization and YACC parsing.
5. What is the role of semantic actions `{ $$ = ... }`?

Conclusion:

The desk calculator using **LEX + YACC** correctly evaluates arithmetic expressions based on a formally defined grammar.
Lex performs lexical analysis, while YACC parses using operator precedence and associativity rules, producing efficient and unambiguous evaluation.

EXPERIMENT NO: **LAB6** **DATE: 8 NOVEMBER 2025**

EXPERIMENT: **Implementation of Left Recursion Elimination in a Grammar**

IMPLEMENTATION: **C++, PYTHON, AND LEX**

Objective

To implement a program that detects and removes **left recursion** from a given context-free grammar (CFG), transforming it into an equivalent grammar that can be parsed by **top-down parsers** (e.g., Recursive Descent Parser).

Theory

In compiler design, **left recursion** occurs when a non-terminal symbol calls itself on the **leftmost side** of a production rule.

A grammar is **left-recursive** if it has a non-terminal `A` such that:

$A \rightarrow A\alpha \mid \beta$

where

- `A` is a non-terminal,
- α is a sequence of grammar symbols (may be terminals or non-terminals),
- β does not begin with `A`.

Left recursion causes **infinite recursion** in top-down parsers (like recursive-descent or LL(1) parsers), so it must be **eliminated**.

Types of Left Recursion

1. **Immediate Left Recursion:**
   Occurs when a non-terminal directly calls itself.
   Example:

   $A \rightarrow A\alpha \mid \beta$

After elimination:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$

2. **Indirect Left Recursion:**
   Occurs when left recursion occurs through a chain of non-terminals.
   Example:

   $A \rightarrow B\alpha$
   $B \rightarrow A\beta \mid \gamma$

Must be resolved by reordering and substituting productions.

Algorithm

Immediate Left Recursion Removal Algorithm

Step 1: Input the number of productions and grammar rules.

Step 2: For each non-terminal A:

Separate productions into two sets:

α = set of right parts starting with A (left-recursive)

β = set of right parts not starting with A (non-left-recursive)

Step 3: If α is not empty:

Create a new non-terminal A'

Rewrite rules:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$

Step 4: Display the modified grammar.

Implementation in C++ Program:

```
#include <iostream>
#include <vector>
#include <sstream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of productions: ";
    cin >> n;
    cin.ignore();

    vector<string> lhs(n), rhs(n);
    cout << "Enter productions (e.g. E->E+T|T):" << endl;
    for (int i = 0; i < n; i++) {
        getline(cin, lhs[i]);
        size_t pos = lhs[i].find("->");
        rhs[i] = lhs[i].substr(pos + 2);
        lhs[i] = lhs[i].substr(0, pos);
    }

    cout << "\nAfter removing left recursion:\n";
    for (int i = 0; i < n; i++) {
        string A = lhs[i];
        string right = rhs[i];
        vector<string> alpha, beta;
        stringstream ss(right);
        string token;

        while (getline(ss, token, '|')) {
            if (token[0] == A[0])
                alpha.push_back(token.substr(1));
            else
                beta.push_back(token);
        }
```

```
        if (alpha.empty()) {
            cout << A << " -> " << right << endl;
        } else {
            string Aprime = A + "'";
            cout << A << " -> ";
            for (size_t j = 0; j < beta.size(); j++) {
                cout << beta[j] << Aprime;
                if (j != beta.size() - 1) cout << " | ";
            }
            cout << endl;

            cout << Aprime << " -> ";
            for (size_t j = 0; j < alpha.size(); j++) {
                cout << alpha[j] << Aprime << " | ";
            }
            cout << "ε" << endl;
        }
    }
    return 0;
}
```

Implementation in Python:

```
def remove_left_recursion(grammar):
    lhs, rhs = grammar.split("->")
    alternatives = rhs.split("|")
    alpha = []
    beta = []

    for alt in alternatives:
        if alt.startswith(lhs):
            alpha.append(alt[len(lhs):])
        else:
            beta.append(alt)

    if not alpha:
        print(grammar)
        return

    new_nonterm = lhs + "'"
    print(f"{lhs} -> {' | '.join(b + new_nonterm for b in beta)}")
    print(f"{new_nonterm} -> {' | '.join(a + new_nonterm for a in
alpha)} | ε")

# Example
g = input("Enter production (e.g. E->E+T|T): ")
remove_left_recursion(g)
```

Implementation in Lex/Flex:

```
%{
#include <stdio.h>
%}

%%
[A-Z]->[A-Z][^|]*\|.*   { printf("Grammar detected with possible left
recursion: %s\n", yytext); }
.|\n                    ;
%%

int main() {
    printf("Enter grammar (e.g. E->E+T|T):\n");
    yylex();
    return 0;
}
```

Sample Input:

```
E->E+T|T
T->T*F|F
F->(E)|id
```

Sample Output:

```
After removing left recursion:
E -> T E'
E' -> +T E' | ε
T -> F T'
T' -> *F T' | ε
F -> (E) | id
```

Conclusion

This experiment demonstrates the **removal of left recursion** from context-free grammars, which is crucial for constructing **top-down parsers** such as **recursive descent parsers**.
The implementation in **C++** and **Python** provides practical insight into:

- Grammar representation and parsing
- Separation of left-recursive and non-left-recursive productions
- Grammar rewriting for parser compatibility

Thus, the resulting grammar becomes suitable for LL(1) parsing and ensures termination of recursive-descent algorithms.

EXPERIMENT NO: **LAB7** **DATE: 15 NOVEMBER 2025**

EXPERIMENT: **Implementation of Left Factoring in a Grammar**

IMPLEMENTATION: **C++, PYTHON, AND LEX**

Objective

To design and implement a program using **C++** and **Lex** to perform **Left Factoring** on a given context-free grammar (CFG) so that the grammar becomes suitable for **LL(1) predictive parsing**.

2. Theory

✓ What is Left Factoring?

Left Factoring is a grammar transformation technique used when two or more production rules of the same non-terminal begin with a **common prefix**.

Example:

$A \rightarrow \alpha\beta1 \mid \alpha\beta2$

Both alternatives begin with α.

✓ Left-Factored Form:

$A \rightarrow \alpha A'$
$A' \rightarrow \beta1 \mid \beta2$

Why Left Factoring?

- Removes ambiguity in production selection
- Makes grammar LL(1)-compatible
- Helps building predictive parsing tables
- Eliminates common prefixes so parser can decide correctly

---

3. Algorithm

Algorithm for Left Factoring

Step 1: Input the grammar production of form $A \rightarrow \alpha1 \mid \alpha2 \mid \ldots \mid \alpha n$
Step 2: Group all productions of A.
Step 3: Check for common prefix among all αi.
Step 4: Extract the longest common prefix α.
Step 5: Create a new non-terminal A'.
Step 6: Rewrite production as:
A $\rightarrow \alpha A'$
A' $\rightarrow \beta1 \mid \beta2 \mid ... \mid \beta n$ (β = αi without prefix α)
Step 7: If no common prefix exists, leave rule unchanged.
Step 8: Print the left-factored grammar.

Implementation in C++:

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
using namespace std;

// Function to find longest common prefix
string commonPrefix(vector<string> v) {
    if (v.empty()) return "";
    string prefix = v[0];
    for (int i = 1; i < v.size(); i++) {
        string s = v[i];
        int j = 0;
        while (j < prefix.size() && j < s.size() && prefix[j] == s[j])
            j++;
        prefix = prefix.substr(0, j);
    }
    return prefix;
}

int main() {
    string production;
    cout << "Enter Production (e.g. S->abX|abY|abZ|a): ";
    getline(cin, production);

    // Split LHS and RHS
    int pos = production.find("->");
    string lhs = production.substr(0, pos);
    string rhs = production.substr(pos + 2);

    // Split RHS by '|'
    stringstream ss(rhs);
    vector<string> prods;
    string temp;
    while (getline(ss, temp, '|'))
        prods.push_back(temp);

    string prefix = commonPrefix(prods);

    cout << "\n=== Left Factored Grammar ===\n";

    if (prefix == "" || prefix.size() == 0) {
        // No factoring needed
        cout << production << endl;
    } else {
        string newNT = lhs + "'";

        cout << lhs << " -> " << prefix << newNT << endl;

        cout << newNT << " -> ";
        for (int i = 0; i < prods.size(); i++) {
            string beta = prods[i].substr(prefix.size());
            if (beta == "") beta = "ε";    // handle empty suffix
            cout << beta;
            if (i != prods.size() - 1)
                cout << " | ";
        }
        cout << endl;
    }

    return 0;
}
```

Implementation in Python:

```
def longest_common_prefix(strings):
    """Find the longest common prefix among all strings."""
    if not strings:
        return ""
    prefix = strings[0]

    for s in strings[1:]:
        temp = ""
        for i in range(min(len(prefix), len(s))):
            if prefix[i] == s[i]:
                temp += prefix[i]
            else:
                break
        prefix = temp

    return prefix


def left_factor(grammar_line):
    """Perform left factoring for a single grammar rule."""
    if "->" not in grammar_line:
        return "Invalid grammar format!"

    lhs, rhs = grammar_line.split("->")
    productions = [p.strip() for p in rhs.split("|")]

    # Find common prefix
    prefix = longest_common_prefix(productions)

    if prefix == "":
        # No left factoring possible
        return f"No Left Factoring Needed:\n{grammar_line}"

    # Create new non-terminal
    new_nt = lhs + "'"

    # Compute beta productions (suffixes)
    betas = []
    for p in productions:
        beta = p[len(prefix):]
        if beta == "":
            beta = "ε"
        betas.append(beta)

    # Prepare output
    result = []
    result.append(f"{lhs} -> {prefix}{new_nt}")
    result.append(f"{new_nt} -> " + " | ".join(betas))

    return "\n".join(result)


# -------- MAIN PROGRAM --------
if __name__ == "__main__":
    print("Enter Grammar Production (e.g., S->abcX|abcY|abcZ):")
    grammar = input().strip()

    print("\n=== Left Factored Grammar ===")
    print(left_factor(grammar))
```

Implementation in Lex (Detection Only):

```
%{
#include <stdio.h>
%}

%%
([A-Z])->([a-zA-Z]+)\|([a-zA-Z]+)  {
    printf("Possible Left Factoring Detected in: %s\n", yytext);
}
.|\n ;
%%

int main() {
    printf("Enter grammar line:\n");
    yylex();
    return 0;
}
```

Sample Input & Output:

Input

```
S->abcX|abcY|abcZ

E->E+T|E-T|T
```

Output

```
=== Left Factored Grammar ===
S -> abcS'
S' -> X | Y | Z

E->E+T|E-T|T
```

Conclusion:

This experiment successfully demonstrates how **Left Factoring** is used to transform grammars into **LL(1)-compatible** form by extracting common prefixes and introducing new non-terminals. It makes grammar deterministic and easier to parse using top-down parsers.

Viva Question Samples:

1. What is left factoring?
2. Why do we need left factoring before predictive parsing?
3. Does left factoring change the language generated?
4. Difference between left recursion removal and left factoring.
5. How do you find the longest common prefix?
